BULK CERTIFICATE GENERATOR

Master Prompt: Planning to Production

A phased, end-to-end guide for building a production-ready Python backend API

How to use: Upload the assignment PDF alongside this document to your AI coding assistant. Follow the phases sequentially and verify each stage before continuing.

Recommended stack: FastAPI · PostgreSQL · SQLAlchemy · Alembic · Celery/Redis · ReportLab · Pytest · Docker · GitHub Actions

Role and Objective

Act as a Senior Python Backend Engineer, Software Architect, DevOps Engineer, QA Engineer, and Technical Mentor.

I want to build a complete, production-ready **Bulk Certificate Generator Backend API** from scratch, following the entire software development lifecycle (SDLC), from requirement analysis and planning to implementation, testing, deployment, and production monitoring.

This is a real backend assignment. Do not just generate sample code or a basic MVP. Help me build a reliable, secure, maintainable, scalable, well-tested application that I can demonstrate during a technical interview and confidently explain, debug, and modify.

1. Project Requirements

Build a backend system that allows an organization to generate certificates for hundreds or thousands of participants using a single bulk request.

The application must:

- Accept bulk certificate generation requests.
- Validate request data and individual recipient information.
- Generate personalized certificates from one predefined template.
- Track the status and progress of each generation job.
- Continue processing other recipients when an individual certificate fails.
- Report successful and failed certificate generations.
- Allow clients to retrieve generated certificates.
- Store job information and generation results in a relational database.
- Provide automated tests and comprehensive documentation.
- Support local development and production deployment.

Follow the requirements in the provided assignment. Clearly distinguish mandatory requirements from optional improvements.

2. Technology Stack

Use the following recommended stack unless there is a strong technical reason to choose otherwise:

- Language: Python 3.12 or another currently supported, compatible Python version.
- Backend framework: FastAPI.
- API server: Uvicorn.
- Database: PostgreSQL.
- ORM: SQLAlchemy 2.x.
- Database migrations: Alembic.
- Request validation: Pydantic.
- Background processing: Celery with Redis, or another appropriate job-processing approach.
- Certificate generation: ReportLab for PDF generation.
- Testing: Pytest and HTTPX.
- Code quality: Ruff, formatting, type checking, and pre-commit where appropriate.
- Authentication: JWT or another suitable authentication mechanism if required by the deployment model.
- Containerization: Docker and Docker Compose.
- CI/CD: GitHub Actions.

- API documentation: OpenAPI/Swagger.
- Deployment: A suitable cloud platform or VPS with managed or containerized services.

Choose compatible, stable dependency versions and document the choices. Avoid adding unnecessary technologies that make the project harder to understand or maintain.

3. Development Rules

- Work in clearly defined phases.
- Complete one phase before proceeding to the next.
- Explain the purpose of each phase in simple language.
- Show the exact folder structure and file paths.
- Provide complete, executable code for every required file.
- Never use placeholders such as “implement this later” for core functionality.
- Do not skip configuration, database migrations, error handling, or tests.
- Explain how each file connects to the rest of the application.
- Provide exact terminal commands for Windows PowerShell and, when useful, Linux/macOS.
- After each phase, give verification steps and expected results.
- Wait for my confirmation before proceeding to the next major phase.
- If I share an error or terminal output, diagnose it using the actual error instead of guessing.
- Keep the code consistent with the previously completed phases.
- Never claim that tests, deployments, security checks, or integrations succeeded unless they were actually executed and verified.
- Keep the implementation simple enough for a fresher to understand and explain in an interview.

Phase 1: Requirement Analysis and Planning

Start by analyzing the assignment. Produce:

1. Functional requirements.
2. Non-functional requirements.
3. User roles and system actors.
4. Assumptions and constraints.
5. Acceptance criteria.
6. System use cases.
7. API endpoint requirements.
8. Data flow from request submission to certificate retrieval.
9. Potential failure scenarios.
10. Mandatory versus optional features.

Create a prioritized development roadmap. Do not write application code until the architecture and requirements are clear.

Phase 2: Architecture and Technical Design

Design a maintainable architecture using appropriate layers:

- API routes/controllers; request and response schemas; business logic/services; database models and repositories where useful; background task workers; certificate generation service; file storage abstraction; configuration and

logging; error handling; automated tests.

Provide an architecture diagram, request lifecycle diagram, database ER diagram, component responsibilities, technology selection rationale, processing strategy and trade-offs, and security and scalability considerations.

Explain why asynchronous background processing is appropriate for large bulk requests and how the system behaves if a worker fails or restarts.

Phase 3: Repository and Environment Setup

Create the complete project structure. Include:

- Python virtual environment and dependency management.
- Environment variables, .env.example, and .gitignore.
- Application entry point and FastAPI configuration.
- PostgreSQL connection and SQLAlchemy setup.
- Alembic initialization.
- Logging configuration and health-check endpoint.
- Docker and Docker Compose configuration.

Give exact commands to install dependencies, start the database, run migrations, start the application, and verify that the API works. Never commit passwords, tokens, private keys, or production secrets.

Phase 4: Database Design

Design normalized database models for generation jobs, recipients or certificate records, individual generation results, certificate storage metadata, and optional audit/event records when justified.

Define appropriate fields, data types, primary keys, foreign keys, indexes, uniqueness rules, timestamps, and relationships. Support job statuses PENDING, PROCESSING, COMPLETED, COMPLETED_WITH_ERRORS, and FAILED.

Define individual certificate statuses separately where appropriate. Create SQLAlchemy models, Pydantic schemas, Alembic migrations, and database constraints.

Explain transactions, retries, duplicate requests, and concurrent workers. Prefer object storage or a suitable local storage abstraction for PDFs, keeping file metadata in the database.

Phase 5: API Contract and Implementation

Implement and document these endpoints:

- POST /api/v1/certificate-jobs — create a bulk generation job.
- GET /api/v1/certificate-jobs/{job_id} — retrieve job status, progress, totals, and success/failure counts.
- GET /api/v1/certificate-jobs/{job_id}/results — retrieve individual results and errors.
- GET /api/v1/certificates/{certificate_id} — retrieve or download an authorized certificate.
- GET /health — check application health.

Add pagination to endpoints that return many results. Use appropriate HTTP status codes, response schemas, validation errors, and consistent error formats. Document realistic request/response JSON. Return a job ID promptly while a large request is processed in the background.

Phase 6: Input Validation and Business Rules

Validate recipient names, email addresses where required, required certificate fields, completion dates, recipient list size, duplicates according to defined rules, missing or malformed fields, request size, identifiers, and state transitions.

Define whether invalid recipients reject the entire request or become individual failures. Keep this behavior consistent. Prevent one invalid recipient from unnecessarily blocking valid recipients. Return actionable validation errors without exposing stack traces.

Phase 7: PDF Certificate Generation

Create one professional predefined PDF certificate template containing recipient name, course/event name, completion date, issuing organization, certificate ID, and optionally a verification URL or QR code.

Keep the certificate service separate from API routes. Ensure correct layout, long-name handling, practical font/Unicode support, safe text handling, unique identifiers, consistent output naming, PDF validation, temporary-file cleanup, and secure storage.

Provide sample generated certificates for testing when the environment supports file creation.

Phase 8: Background Processing and Bulk Reliability

Implement Celery with Redis or justify an alternative. Save the job and recipient records; queue work safely; process recipients independently; record successes and failures; update progress reliably; retry appropriate transient failures; prevent duplicate certificates during retries; recover from worker restarts where possible; handle exceptions; and set the final job status correctly.

Define partial success, complete failure, retry exhaustion, and cancellation behavior if supported. Avoid in-memory progress counters that become incorrect after restarts. Use durable state and suitable transaction/concurrency controls. Document queue guarantees, idempotency, and limitations.

Phase 9: Security and Production Readiness

- Authentication and authorization.
- Job and certificate ownership checks.
- Input size limits and rate limiting.
- Secure file-download authorization.
- Path traversal prevention and safe filenames.
- SQL injection prevention using parameterized ORM operations.
- CORS restrictions and secret management.
- TLS at the deployment boundary.
- Dependency vulnerability checks.
- Safe logging without exposing sensitive recipient information.
- Retention/deletion policies and protection against unauthorized job enumeration.
- Appropriate error responses.

Use secure defaults. Explain which controls are mandatory for public deployment and which depend on authentication requirements.

Phase 10: Automated Testing

Write meaningful tests for startup and health checks; job creation; input validation; database operations; PDF generation; job status/progress; individual failures; partial success; certificate retrieval; unauthorized access; duplicate requests and retries; and background worker errors.

Use isolated test data and an appropriate test database. Mock external storage or queue services only where suitable, and include integration tests for critical workflows. Provide test and coverage commands. Never claim test success without actual execution.

Phase 11: API Documentation and Developer Experience

Create OpenAPI documentation and a comprehensive README covering project overview, features, architecture, stack, prerequisites, local setup, environment variables, migrations, application and worker startup, Docker, endpoints, request/response examples, downloads, tests, troubleshooting, design decisions, and limitations.

Include curl and Windows PowerShell examples. Add a Postman collection if useful.

Phase 12: Docker and Local Integration

Create production-conscious Dockerfiles and Docker Compose for the API, PostgreSQL, Redis, and worker where applicable.

Use environment-driven configuration, persistent database storage, health checks, correct service dependencies, non-root containers where practical, external secrets, accessible logs, and documented restart behavior. Test the complete workflow through Docker.

Phase 13: CI/CD Pipeline

Create a GitHub Actions pipeline for dependency installation, formatting/linting, type checking where configured, unit and integration tests, coverage reporting, Docker image builds, and optional image security scanning.

Explain repository secrets and deployment credentials. Never automatically deploy untested code to production.

Phase 14: Production Deployment

Choose a suitable deployment target based on cost, reliability, and operational complexity. Document infrastructure setup, secrets, PostgreSQL and Redis, certificate storage, image publishing, safe migrations, API and worker deployment, TLS/domain setup, health checks, deployed workflow verification, and rollback/recovery.

Explain how to avoid losing queued jobs or generated certificates during deployments. Keep local and production configuration separate.

Phase 15: Observability and Operations

Implement or document structured logs, request/job IDs, error metrics, queue depth and worker health, job duration and throughput, database health, storage usage, failure alerts, backups/restores, retention/cleanup, and operational troubleshooting.

Explain the most important metrics for processing thousands of certificates.

Phase 16: Performance and Scalability

Benchmark realistic bulk requests. Evaluate batch sizes, concurrent submissions, worker concurrency, database queries, storage throughput, memory, queue delays, and retry behavior.

Apply batching, pagination, indexes, concurrency limits, or storage improvements when measurements justify them. Document actual benchmark results separately from estimates.

Phase 17: Final End-to-End Verification

1. Start infrastructure and apply migrations.
2. Start API and workers.
3. Submit a bulk request and receive a job ID.
4. Check status and observe progress.
5. Verify successful and failed recipient records.
6. Download a certificate and verify its contents.

7. Restart a worker and test recovery.
8. Run automated tests and check the health endpoint.
9. Inspect logs and failure handling.
10. Confirm the README reproduces the setup.

Provide a final acceptance checklist mapping every assignment requirement to implementation and tests.

Phase 18: Interview Preparation and Handover

Prepare me to explain the project as a fresher. Include a two-minute introduction; architecture and database explanations; API lifecycle; reasons for technology choices; partial failures; retry idempotency; progress calculation; file security; scaling and deployment; trade-offs and limitations; common interview questions with simple answers; practical code-change exercises; and project-specific debugging scenarios.

Ensure I understand the code rather than merely memorizing answers.

Required Deliverables

1. Complete working source-code repository structure and application files.
2. Database models and migrations.
3. API schemas and routes.
4. PDF generation implementation.
5. Background worker implementation.
6. Automated tests.
7. Docker configuration.
8. GitHub Actions CI workflow.
9. .env.example and comprehensive README.
10. API examples or Postman collection.
11. Deployment guide.
12. Security and operational checklist.
13. Requirement-to-test traceability matrix.
14. Interview preparation notes.

How to Start

Begin with **Phase 1: Requirement Analysis and Planning**. Analyze the provided Bulk Certificate Generator assignment, identify its exact mandatory requirements, define acceptance criteria, highlight ambiguities, and produce the implementation roadmap.

Then stop and wait for my approval before starting Phase 2.

Do not generate the entire codebase in one unstructured response. Build the application incrementally, validate each phase, and keep implementation decisions consistent from planning through production.